

Topology Optimization
Structural optimization of a truss structure

Tommaso Bocchietti 10740309

A.Y. 2024/25



POLITECNICO
MILANO 1863

Contents

1	Methodology	3
2	Theoretical background	3
2.1	Optimization theory	3
2.2	Structural optimization problem (linear elastic)	4
2.2.1	Compliance minimization problem	5
2.2.2	Volume minimization problem	6
3	Topology optimization of a cantilever beams structure	6
3.1	Requests	6
3.2	Solution procedure	7
3.3	Results	8
4	Topology optimization of a motorcycle frame	10
4.1	Requests	10
4.2	Solution procedure	11
4.3	Results	12
A	Compliance minimization flowchart	15
B	MATLAB code for the compliance minimization algorithm	15

List of Figures

1	Unloaded truss structure to be optimized	6
2	Optimized truss structure for Case 1 ($\alpha = 0.05$)	8
3	Optimized truss structure for Case 2 ($\alpha = 0.10$)	8
4	Optimized truss structure for Case 3 ($\alpha = 0.15$)	9
5	Optimized truss structure for Case 4 ($\alpha = 0.20$)	9
6	Unloaded motorcycle frame to be optimized	10
7	Optimized motorcycle frame for $w_1 = 0.5$ and $w_2 = 0.5$	12
8	Optimized motorcycle frame for $w_1 = 0.0$ and $w_2 = 1.0$	13
9	Optimized motorcycle frame for $w_1 = 0.2$ and $w_2 = 0.8$	13
10	Optimized motorcycle frame for $w_1 = 0.8$ and $w_2 = 0.2$	14
11	Optimized motorcycle frame for $w_1 = 1.0$ and $w_2 = 0.0$	14
12	Compliance minimization flowchart	15

List of Tables

1	Optimization cases	6
2	Cross-sectional areas and stresses for elements [64, 77, 82]	10
3	Braking load case	11
4	Acceleration load case	11
5	Different values of w_1 and w_2	12

Listings

1	Compliance and sensitivities computation for two load cases	11
2	MATLAB code for the compliance minimization algorithm	15

1 Methodology

The aim of this work is to optimize the topology of a truss structure, given a set of constraints and a set of loads.

In the following sections, we will at first give an overview of the topology optimization problem, describing the main concepts and the mathematical foundations needed.

Then, we will present a couple of examples of truss structures, showing how the optimization problem can be formulated and solved in practice. In particular, we will show the results obtained using our custom implementation of CONLIN algorithm for the optimization of:

1. Cantilever beam structure (single load case)
2. Motorcycle frame structure (multiple load case)

2 Theoretical background

In this section, we will provide a brief overview of the theoretical background needed to understand the solution to the topology optimization problems described in Sections 3 and 4.

For a more in-depth analysis of the topic, we suggest consulting the references provided at the end of this document.

2.1 Optimization theory

At its core, a topology optimization problem is nothing more than a mathematical optimization problem. The goal is to find the minimum or maximum of a certain function (the objective function) subject to a set of constraints. In mathematical terms, the general optimization problem can be formulated as follows:

$$(P) \quad \begin{cases} \min_{\mathbf{x}} & f(\mathbf{x}) \\ \text{s. t.} & g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m, \\ & h_j(\mathbf{x}) = 0, \quad j = 1, \dots, n, \\ & \mathbf{x} \in \Omega \end{cases} \quad (1)$$

$$\Omega := \{\mathbf{x} \in \mathbb{R}^n \mid x_j^{\min} \leq x_j \leq x_j^{\max}, \quad j = 1, \dots, n\}$$

Where $\mathbf{x}$ is the design variable, $f(\mathbf{x})$ is the objective function, $g_i(\mathbf{x})$ are inequality constraints, $h_j(\mathbf{x})$ are equality constraints, and Ω is the design domain.

The problem depicted in Equation 1 can in general be hard to solve, especially when the objective function and constraints are non-linear and the design domain is large. Also, the number of possible solutions can be infinite (e.g., multiple local minima or maxima), thus making the optimization problem even more challenging.

Lagrangian's duality In theory, one can solve the optimization problem using direct substitution between the objective function and the constraints to find the optimal solution. However, in practice, this is almost impossible, especially for complex problems with many constraints and design variables.

In order to solve constrained optimization problems, the Lagrangian function is commonly used. The Lagrangian function is defined as follows:

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \sum_{i=1}^m \lambda_i g_i(\mathbf{x}) \quad (2)$$

Where $\boldsymbol{\lambda}$ are the Lagrange multipliers associated with the inequality constraints. The Lagrangian function allows us to convert the constrained optimization problem into an unconstrained optimization problem by introducing the Lagrange multipliers.

Leveraging the Lagrangian function, the optimization problem (P) in Equation 1 can be reformulated as follows:

$$(D) \quad \begin{cases} \max_{\boldsymbol{\lambda}} \phi(\boldsymbol{\lambda}) = \max_{\boldsymbol{\lambda}} (\min_{\mathbf{x} \in \Omega} \mathcal{L}(\mathbf{x}, \boldsymbol{\lambda})) \\ \text{s. t.} & \boldsymbol{\lambda} \geq 0 \end{cases} \quad (3)$$

The optimization problem (D) in Equation 3 is called the dual optimization problem, and it's of particular interest given that if we are able to find $\boldsymbol{\lambda}^*$ solution to (D), then we can find the optimal solution to the original optimization problem (P) by solving the following equation:

$$\mathbf{x}^* = \operatorname{argmin}_{\mathbf{x} \in \Omega} \mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}^*) \quad (4)$$

The dual optimization problem (D) is usually easier to solve than the primal optimization problem (P), as it is an unconstrained optimization problem.

Sequential Linear Programming (SLP) One common approach to solving non-linear optimization problems is to use a linear (convex) approximation of the objective function and constraints. To do so, the sequential linear programming (SLP) method is often used.

$$(P_{SLP}) \quad \begin{cases} \min_{\mathbf{x}} & f(\mathbf{x}^{(k)}) + \nabla f(\mathbf{x}^{(k)})^T (\mathbf{x} - \mathbf{x}^{(k)}) \\ \text{s. t.} & g_i(\mathbf{x}^{(k)}) + \nabla g_i(\mathbf{x}^{(k)})^T (\mathbf{x} - \mathbf{x}^{(k)}) \leq 0, \quad i = 1, \dots, m, \\ & h_j(\mathbf{x}^{(k)}) + \nabla h_j(\mathbf{x}^{(k)})^T (\mathbf{x} - \mathbf{x}^{(k)}) = 0, \quad j = 1, \dots, n, \\ & \mathbf{x} \in \Omega \end{cases} \quad (5)$$

$$\Omega := \{\mathbf{x} \in \mathbb{R}^n \mid x_j^{min} \leq x_j \leq x_j^{max}, -l_j^k \leq x_j - x_j^k \leq u_j^k, \quad j = 1, \dots, n\}$$

Where the superscript (k) denotes the iteration number, and l_j^k and u_j^k are the lower and upper bounds of the design variables at iteration k (move limits).

The SLP method iteratively solves a sequence of linear approximations of the original optimization problem, updating the design variables at each iteration.

Convex linearization (CONLIN) When dealing with structural optimization problems, a custom linearization method called Convex Linearization (CONLIN) built on top of the SLP method has found to be particularly efficient.

In particular, if we assume all the design variables x_j to be strictly positive, the approximation of the objective function and constraints based on CONLIN became a convex and separable functions of the design variables. To obtain that formulation, we introduce a set of intervening variables y_j that depend on the design variables x_j as follows:

$$y_j(x_j) = \begin{cases} x_j & \text{if } \frac{\partial g_i(\mathbf{x}^{(k)})}{\partial x_j} > 0 \\ \frac{1}{x_j} & \text{if } \frac{\partial g_i(\mathbf{x}^{(k)})}{\partial x_j} \leq 0 \end{cases} \quad (6)$$

The objective function (and similarly also constraints) can then be approximated as follows:

$$\begin{aligned} f^{(C,k)}(\mathbf{x}) &\approx f(\mathbf{x}^{(k)}) + \sum_{j=1}^n \frac{\partial f(\mathbf{x}^{(k)})}{\partial y_j} (y_j(x_j) - y_j(x_j^{(k)})) \\ &= f(\mathbf{x}^{(k)}) + \sum_{j \in \Omega_+} \frac{\partial f(\mathbf{x}^{(k)})}{\partial x_j} (x_j - x_j^{(k)}) + \sum_{j \in \Omega_-} \frac{\partial f(\mathbf{x}^{(k)})}{\partial x_j} \frac{x_j^{(k)}(x_j - x_j^{(k)})}{x_j} \end{aligned} \quad (7)$$

Where Ω_+ and Ω_- are the sets of design variables x_j for which the partial derivative of the constraints is positive or negative, respectively.

Notice also that Lagrangian duality can be applied to the CONLIN approximation, leading to a dual optimization problem that is easier to solve than the original optimization problem.

2.2 Structural optimization problem (linear elastic)

Based on the general optimization problem described in Section 2.1, we can now define the structural optimization problem for a linear elastic structure.

Considering a non-singular K matrix, we can define the nested formulation of the structural optimization problem as follows:

$$(SO)_{sf} \quad \begin{cases} \min_{\mathbf{x}} & f(\mathbf{x}, \mathbf{u}(\mathbf{x})) \\ \text{s. t.} & g_i(\mathbf{x}, \mathbf{u}(\mathbf{x})) \leq 0, \quad i = 1, \dots, m \\ & \mathbf{x} \in \Omega \end{cases} \quad (8)$$

$$\Omega := \{\mathbf{x} \in \mathbb{R}^n \mid x_j^{min} \leq x_j \leq x_j^{max}, \quad j = 1, \dots, n\}$$

Notice that the dependence of the displacement field $\mathbf{u}(\mathbf{x})$ on the design variable $\mathbf{x}$ is derived from linear elasticity hypothesis as:

$$\mathbf{u}(\mathbf{x}) = K^{-1}(\mathbf{x})\mathbf{F}(\mathbf{x}) \quad (9)$$

In both the equations above, the design variables $\mathbf{x}$ can represent the material properties of the structure, such as the Young's modulus or the density of the material at each element of the structure, or the geometry of the structure, such as the cross-sectional area of the elements or the thickness of the elements.

In Equation 8, we still haven't defined the objective function $f(\mathbf{x}, \mathbf{u}(\mathbf{x}))$ and the constraints $g_i(\mathbf{x}, \mathbf{u}(\mathbf{x}))$. In the following, we will define two common structural optimization problems: the compliance minimization problem and the volume minimization problem.

2.2.1 Compliance minimization problem

One common type of structural optimization problem, is the minimization of the compliance having a volume constraint.

Compliance is a measure of strain energy stored in a structure, and can be computed from definition as:

$$C_j = \frac{1}{2} \int \delta_j \varepsilon_j dV_j = \frac{1}{2} \mathbf{u}_j^T \mathbf{K}_j \mathbf{u}_j \quad (10)$$

The summation of the compliance of all the elements in the structure gives the total compliance of the structure:

$$C = \sum_{j=1}^n C_j = \frac{1}{2} \mathbf{u}^T \mathbf{K} \mathbf{u} \approx \mathbf{F}^T \mathbf{u} \quad (11)$$

By doing so, the compliance minimization problem can be formulated as follows:

$$(P)_{nf} \quad \begin{cases} \min_{\mathbf{x}} & \mathbf{F}^T \mathbf{u} \\ \text{s. t.} & \sum_{j=1}^n V_j - V_{max} \leq 0 \\ & \mathbf{x} \in \Omega \end{cases} \quad (12)$$

$$\Omega := \{\mathbf{x} \in \mathbb{R}^n \mid x_j^{min} \leq x_j \leq x_j^{max}, \quad j = 1, \dots, n\}$$

One can also proof that the compliance minimization problem is a convex optimization problem, and thus it can be solved efficiently using convex optimization algorithms such as Sequential Linear Programming (SLP) or Convex Linearization (CONLIN) as described in Section 2.1.

In particular, if we suppose to adopt CONLIN approximation to solve the compliance minimization problem, we obtain the following linear approximation of $(P)_{nf}$ of Equation 12:

$$(P)_{nf}^C \quad \begin{cases} \min_{\mathbf{x}} & f^{(C,k)} = f(\mathbf{x}^{(k)}) + \sum_{j \in \Omega_-} \frac{\partial f(\mathbf{x}^{(k)})}{\partial x_j} \frac{x_j^{(k)}(x_j - x_j^{(k)})}{x_j} \\ \text{s. t.} & \sum_{j=1}^n l_j x_j - V_{max} \leq 0 \\ & \mathbf{x} \in \Omega \end{cases} \quad (13)$$

$$\Omega := \{\mathbf{x} \in \mathbb{R}^n \mid x_j^{min} < x_j \leq x_j^{max}, \quad j = 1, \dots, n\}$$

Notice that the approximation of the objective function in Equation 13 is based on the fact that the derivative of the compliance function is always negative, and thus only the Ω_- set of design variables is considered in the approximation:

$$\frac{\partial f(\mathbf{x})}{\partial x_j} = \frac{\partial \mathbf{F}^T \mathbf{u}}{\partial x_j} = -\mathbf{u}^T \frac{\partial \mathbf{K}}{\partial x_j} \mathbf{u} = -\mathbf{u}^T \mathbf{K}_j^0 \mathbf{u} < 0 \quad (14)$$

Based on Equation 13, we can also define the Lagrangian function $\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda})$ as follows:

$$\mathcal{L}^k(\mathbf{x}, \boldsymbol{\lambda}) = f^{(C,k)}(\mathbf{x}) + \lambda \left(\sum_{j=1}^n l_j x_j - V_{max} \right) \quad (15)$$

By doing so, we can define the dual optimization problem $(D)_{nf}$ as follows:

$$(D)_{nf} \quad \begin{cases} \max_{\boldsymbol{\lambda}} & \min_{\mathbf{x} \in \Omega} \mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) \\ \text{s. t.} & \boldsymbol{\lambda} \geq 0 \end{cases} \quad (16)$$

Where, an explicit solution to the dual optimization problem $(D)_{nf}$ can be found by solving the following equation:

$$\frac{\partial \mathcal{L}^k}{\partial x_j} = 0 \quad (17)$$

2.2.2 Volume minimization problem

Another common type of structural optimization problem is the minimization of the volume having a compliance constraint.

Similarly to the compliance minimization problem, the volume minimization problem can be formulated as follows:

$$(P)_{nf} \quad \begin{cases} \min_{\mathbf{x}} & \mathbf{l}^T \mathbf{x} \\ \text{s. t.} & \mathbf{F}^T \mathbf{u} - C_{max} \leq 0 \\ & \mathbf{x} \in \Omega \end{cases} \quad (18)$$

$$\Omega := \{\mathbf{x} \in \mathbb{R}^n \mid x_j^{min} \leq x_j \leq x_j^{max}, \quad j = 1, \dots, n\}$$

3 Topology optimization of a cantilever beams structure

3.1 Requests

The goal of this section is to perform a topology optimization of the truss structure shown in Figure 1, minimizing the compliance C by acting on the cross-sectional areas of the beams. A maximum volume constraint $V_{structure} \leq V_0$ is also imposed.

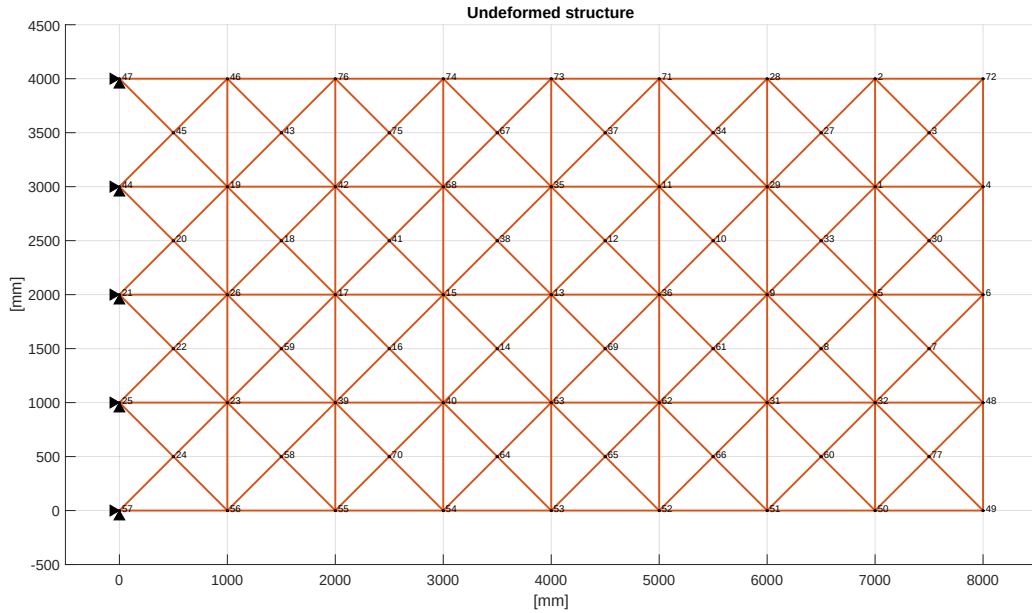


Figure 1: Unloaded truss structure to be optimized

As can be observed, the structure is composed of $N_{nodes} = 77$ nodes and $N_{beams} = 200$ beams. Node indexes are indicated in the figure by the numbers reported besides each node, while the black triangles connected to some nodes represent the applied constraints. In particular, both vertical and horizontal constraints are imposed on nodes 47, 44, 21, 25, 57.

For the case at hand, we suppose to have a vertical load $F = -6000N$ applied to node 49 (lower right corner of the structure), and the beam's Young's modulus to be $E = 210GPa$ (steel). Given these data, we are asked to perform the optimization for the cases reported in Table 1.

Case	α	LB [mm^2]	UB [mm^2]
1	0.05	0.0001	800
2	0.10	0.0001	800
3	0.15	0.0001	800
4	0.20	0.0001	800

Table 1: Optimization cases

Notice that in Table 1, the abbreviations LB and UB stand for Lower Bound and Upper Bound, respectively.

The parameter α is the volume fraction of the structure, defined as $\alpha = \frac{V_0}{V_{max}}$, where $V_{max} = \text{UB} \sum_{j=1}^{N_{beams}} l_j$ is the maximum volume of the structure.

3.2 Solution procedure

At first, we recognize that the given optimization problem is a compliance minimization problem, and so we can use the theoretical background presented in Section 2.2.1.

In addition to what has already been discussed in Section 2.2.1, we adopt some additional naming conventions that will also be used in the code implementation. In particular, in Equation 13, we define the following quantities:

$$\begin{aligned} f^{(C,k)} &= f(\mathbf{x}^{(k)}) + \sum_{j \in \Omega_-} \frac{\partial f(\mathbf{x}^{(k)})}{\partial x_j} \frac{x_j^{(k)}(x_j - x_j^{(k)})}{x_j} \\ &= r_0^k + \sum_{j \in \Omega_-} \frac{q_{0j}^k}{x_j} \end{aligned} \quad (19)$$

Where:

$$\begin{aligned} r_0^k &= f(\mathbf{x}^{(k)}) - \sum_{j \in \Omega_-} x_j^k u_j^T(\mathbf{x}^k) \mathbf{K}_j^0 u_j^T(\mathbf{x}^k) \\ q_{0j}^k &= u_j^T(\mathbf{x}^k) \mathbf{K}_j^0 u_j^T(\mathbf{x}^k) (x_j^k)^2 \end{aligned} \quad (20)$$

Based on these definitions, we can redefine both the optimization problem $(P)_{nf}$ and the Lagrangian function $\mathcal{L}^k(\mathbf{x}, \boldsymbol{\lambda})$ as follows:

$$(P)_{nf}^C \quad \begin{cases} \min_{\mathbf{x}} & f^{(C,k)} = r_0^k + \sum_{j \in \Omega_-} \frac{q_{0j}^k}{x_j} \\ \text{s. t.} & \sum_{j=1}^n l_j x_j - V_{max} \leq 0 \\ & \mathbf{x} \in \Omega \end{cases} \quad (21)$$

$$\Omega := \{\mathbf{x} \in \mathbb{R}^n \mid x_j^{min} < x_j \leq x_j^{max}, \quad j = 1, \dots, n\}$$

$$\mathcal{L}^k(\mathbf{x}, \boldsymbol{\lambda}) = r_0^k + \sum_{j \in \Omega_-} \left(\frac{q_{0j}^k}{x_j} + \lambda_j l_j x_j \right) - \lambda V_{max} \quad (22)$$

Also, the solution of the dual optimization problem $(D)_{nf}$ can be found by solving the following equation:

$$\frac{\partial \mathcal{L}^k}{\partial x_j} = -\frac{q_{0j}^k}{x_j^2} + \lambda_j l_j = 0 \rightarrow x_j^* = \sqrt{\frac{q_{0j}^k}{\lambda_j l_j}} \quad (23)$$

Notice however that the solution given by the equation above may not respect the constraints $x_j^{min} < x_j < x_j^{max}$. In this case, we can use the following projection operator to find the optimal solution:

$$x_j^{(k+1)}(\lambda) = \begin{cases} x_j^* & \text{if } x_j^* \in [x_j^{min}, x_j^{max}] \\ x_j^{min} & \text{if } x_j^* < x_j^{min} \\ x_j^{max} & \text{if } x_j^* > x_j^{max} \end{cases} \quad (24)$$

Finally, we can use the following update rule to find the optimal value of the Lagrangian multiplier λ :

$$\frac{\partial \psi^k(\lambda)}{\partial \lambda} = \sum_{j \in \Omega_-} l_j x_j^{(k+1)} - V_{max} = 0 \quad (25)$$

So basically, one can solve for the Lagrangian multiplier λ by finding numerically the root of the equation above. Once the optimal value of λ is found, we can update the design variables $\mathbf{x}$ using the projection operator defined above, and then update the Lagrangian function $\mathcal{L}^k(\mathbf{x}, \boldsymbol{\lambda})$. This iterative procedure can be repeated until convergence is reached.

One can refer to the MATLAB code provided in the Appendix 2 for the implementation of the solution procedure described above.

3.3 Results

Based on the solution procedure described in Section 3.2, the results of the topology optimization of the truss structure for the cases reported in Table 1 are shown in Figures 2, 3, 4, and 5.

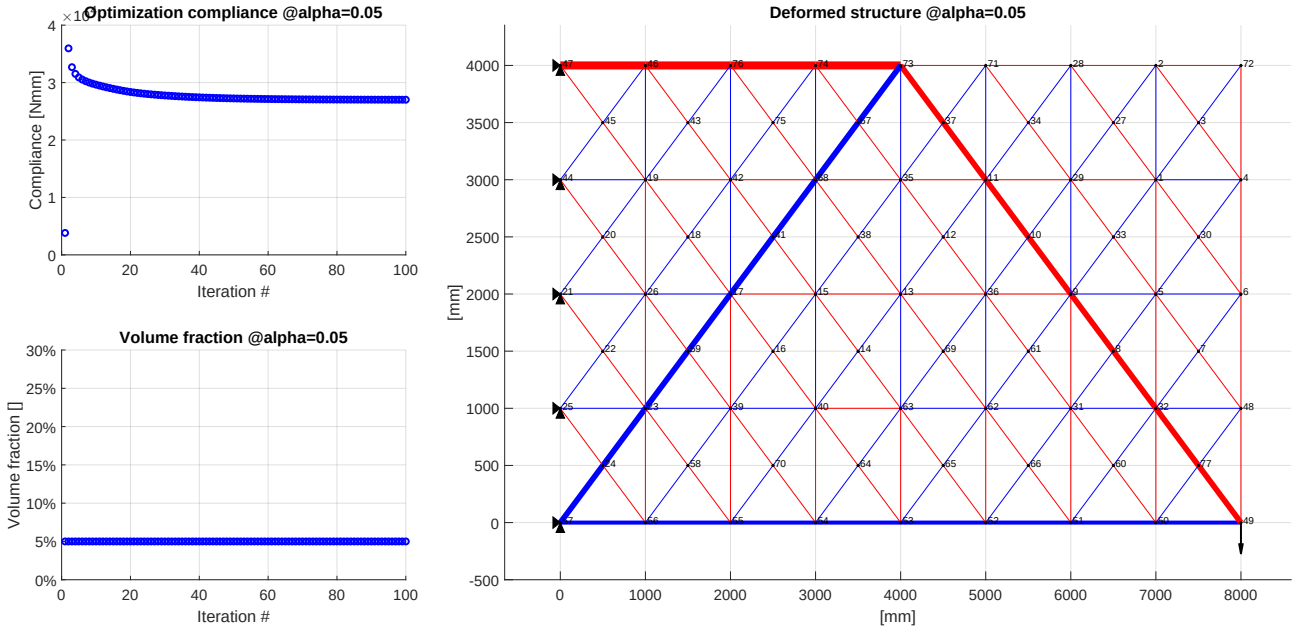


Figure 2: Optimized truss structure for Case 1 ($\alpha = 0.05$)

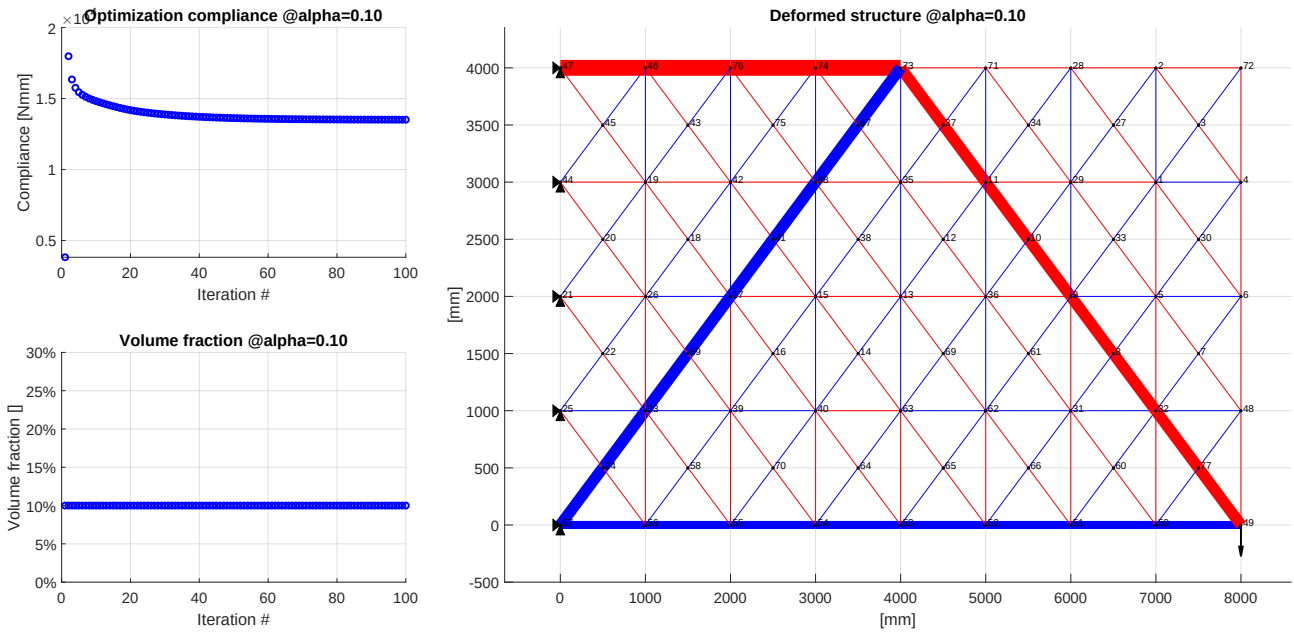


Figure 3: Optimized truss structure for Case 2 ($\alpha = 0.10$)

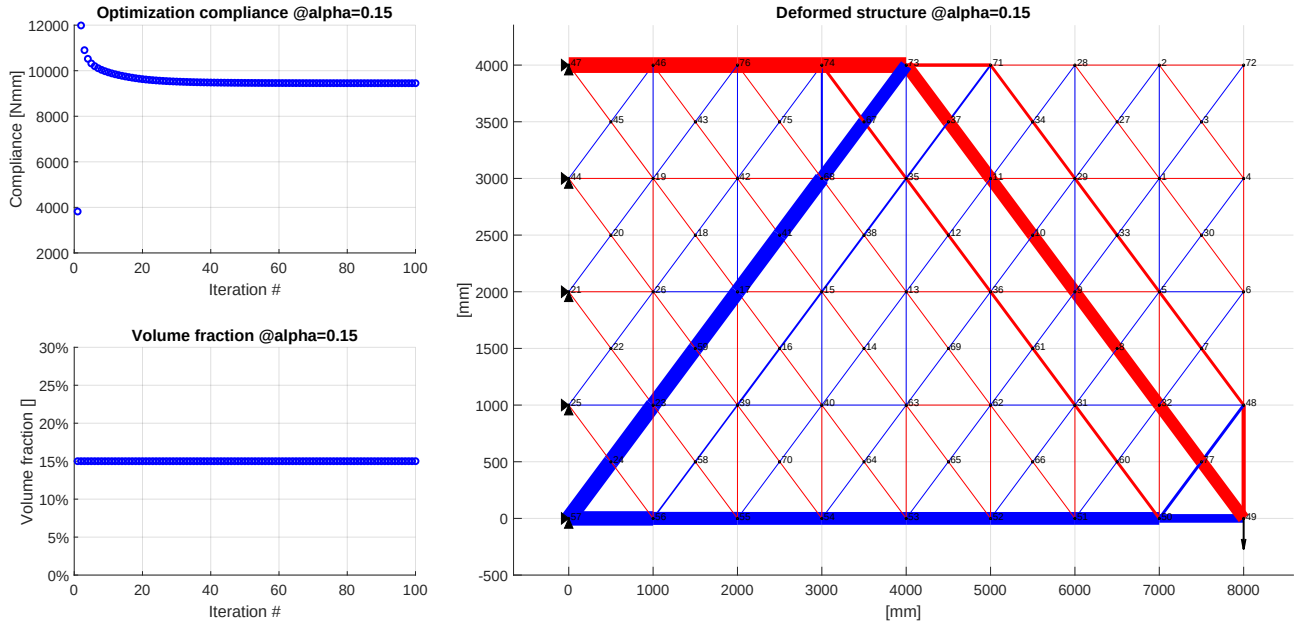


Figure 4: Optimized truss structure for Case 3 ($\alpha = 0.15$)

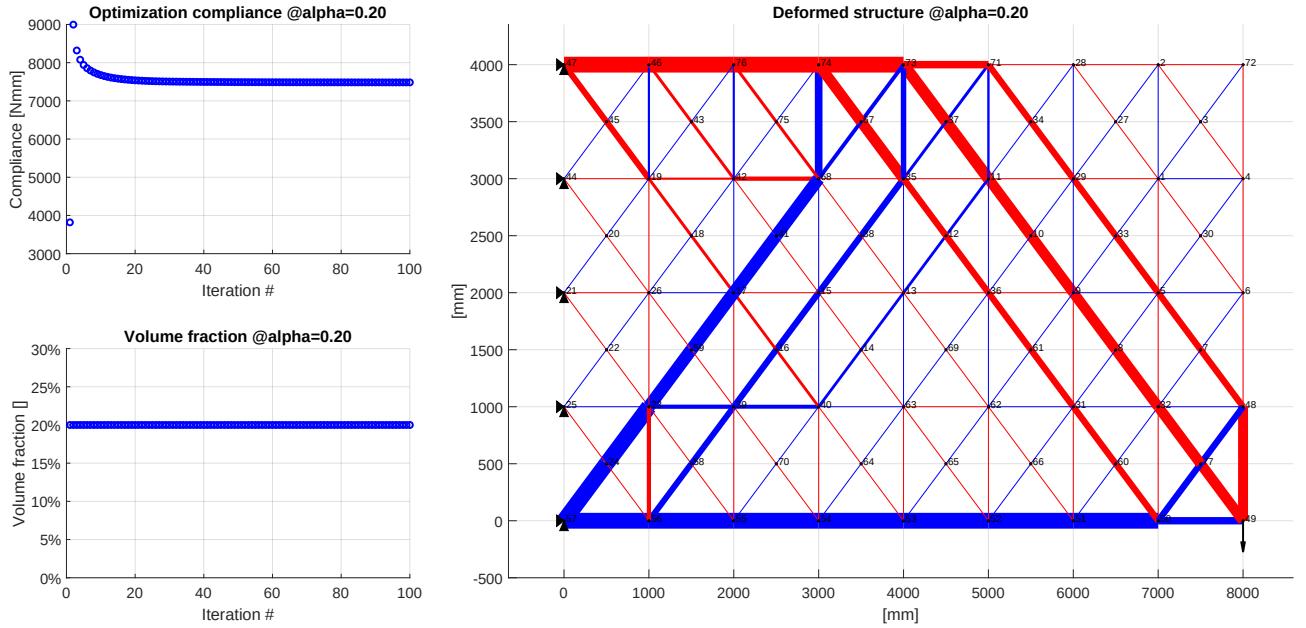


Figure 5: Optimized truss structure for Case 4 ($\alpha = 0.20$)

One can clearly notice how the volume fraction α affects the topology of the optimized structure. In particular, larger values of α lead to a denser structure, with more beams contributing to the load-bearing capacity of the structure. On the other hand, smaller values of α lead to a stiffer structure, with increased compliance values. Finally, as requested by the project, we report in the following table the cross-sectional areas and the stresses acting in elements [64, 77, 82] for the cases reported in Table 1.

Case	Element	Area [mm ²]	Stress [MPa]
1	64	406.27	+29.54
	77	203.14	-29.54
	82	287.27	-29.54
2	64	800.00	+15.00
	77	408.36	-14.69
	82	577.50	-14.69
3	64	800.00	+14.42
	77	653.68	-9.18
	82	800.00	-9.31
4	64	800.00	+13.17
	77	800.00	-7.50
	82	800.00	-8.02

Table 2: Cross-sectional areas and stresses for elements [64, 77, 82]

From Table 2, it can be observed that when the cross-sectional areas of a given element remain within the optimization constraint bounds (i.e., lower bound (LB) and upper bound (UB)), the stresses acting in those elements become equal. This phenomenon occurs because the Lagrangian multiplier is directly proportional to the square stress in cases where the element area stays within these specified bounds. That is:

$$\lambda_j = \frac{\sigma_j^2}{E} \rightarrow \sigma_j = \sqrt{E\lambda_j} \quad (26)$$

4 Topology optimization of a motorcycle frame

4.1 Requests

The goal of this section is to perform a topology optimization of a motorcycle frame shown in Figure 6, minimizing the compliance C by acting on the cross-sectional areas of the beams. A maximum volume constraint $V_{structure} \leq V_0$ is also imposed.

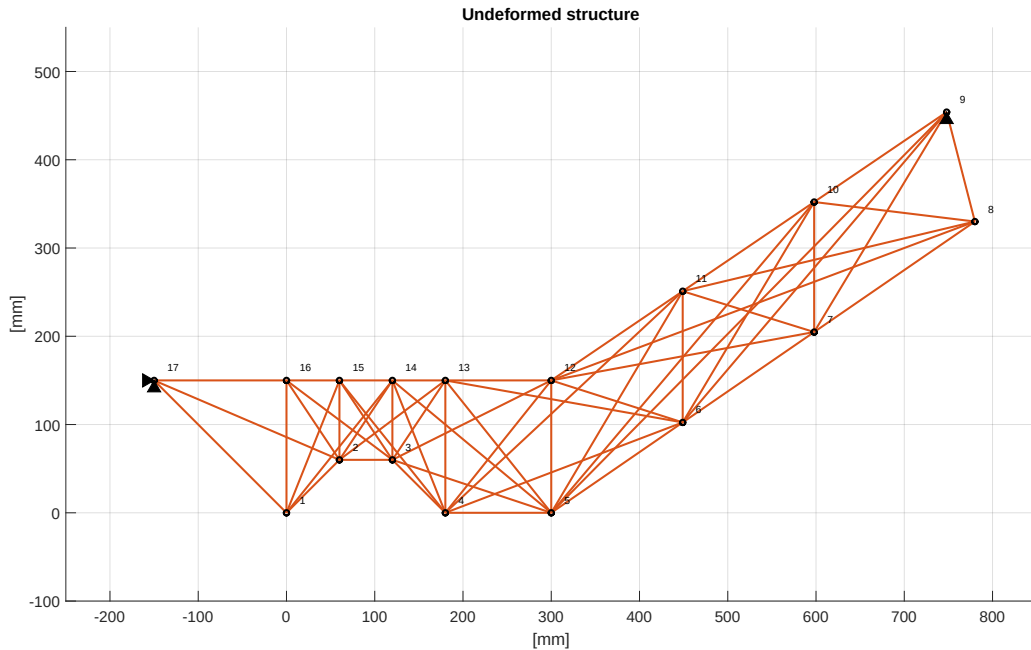


Figure 6: Unloaded motorcycle frame to be optimized

As can be observed, the structure is composed of $N_{nodes} = 17$ nodes and $N_{beams} = 55$ beams. Node indexes are indicated in the figure by the numbers reported besides each node, while the black triangles connected to some nodes represent the applied constraints. In particular, vertical constraints are imposed on nodes 9, 17, while horizontal constraints are also imposed on node 17.

For the case at hand, we suppose to optimize the structure simultaneously for the two most severe load conditions of such frame, which are braking and acceleration. Data for the two load cases are reported in Table 3 and Table 4, respectively.

Node	Load [N]	Direction
8	13000	$-x$
	2000	$-y$
9	9000	$+x$

Table 3: Braking load case

Node	Load [N]	Direction
1	2000	$+x$
	8000	$-y$

Table 4: Acceleration load case

Material properties are the same as in the previous case, with $E = 210GPa$ (steel). Optimization parameters are $\alpha = 0.1$, $LB = 0.001[mm^2]$ and $UB = 700[mm^2]$.

4.2 Solution procedure

The solution procedure is the same as the one described in Section 3.2, with the only difference that we have to consider two load cases simultaneously.

The compliance C is computed as the sum of the compliance for each load case, weighted by the corresponding weight factor.

$$C = w_1 \cdot C_1 + w_2 \cdot C_2 \quad (27)$$

Similarly, the sensitivities are computed as the sum of the sensitivities for each load case, weighted by the corresponding weight factor.

$$\frac{\partial C}{\partial x_i} = w_1 \cdot \frac{\partial C_1}{\partial x_i} + w_2 \cdot \frac{\partial C_2}{\partial x_i} \quad (28)$$

At the implementation level, one can simply compute the compliance and sensitivities for each load case separately and then sum them up, as shown in the following code snippet.

```

1 % FEM Analysis
2 structure_brk = structure_brk.runFEM();
3 structure_acc = structure_acc.runFEM();
4 results.C(k) = w1 * structure_brk.F' * structure_brk.U + ...
5               w2 * structure_acc.F' * structure_acc.U;
6
7 % Sensitivity Analysis
8 for ii = 1:length(structure_brk.elements)
9
10     k01 = structure_brk.elements(ii).getStiffnessMatrix / structure_brk.elements(ii).A;
11     ue1 = structure_brk.U(structure_brk.getDOfidxs(structure_brk.elements(ii)));
12
13     k02 = structure_acc.elements(ii).getStiffnessMatrix / structure_acc.elements(ii).A;
14     ue2 = structure_acc.U(structure_acc.getDOfidxs(structure_acc.elements(ii)));
15
16     results.dCdA(k, ii) = w1 * (- ue1' * k01 * ue1) + ...
17                           w2 * (- ue2' * k02 * ue2);
18
19 end

```

Listing 1: Compliance and sensitivities computation for two load cases

4.3 Results

At first, as requested in Section 4.1, the optimization is run for $w_1 = 0.5$ and $w_2 = 0.5$, with $\alpha = 0.1$, $LB = 0.001[mm^2]$ and $UB = 700[mm^2]$. The results are shown in Figure 7.

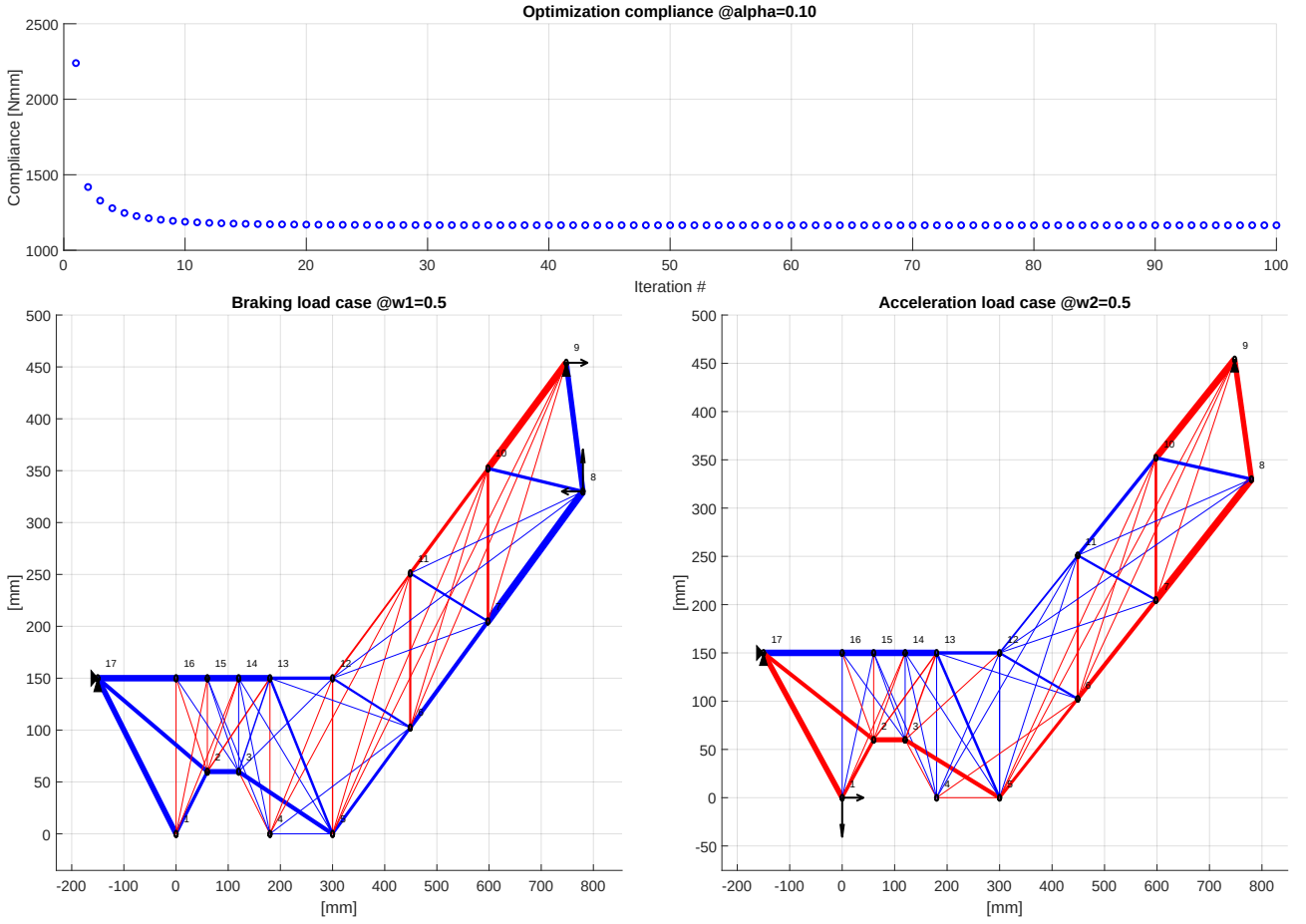


Figure 7: Optimized motorcycle frame for $w_1 = 0.5$ and $w_2 = 0.5$

We now proceed showing the results for different values of w_1 and w_2 . In particular, we consider the following cases:

Case	w_1	w_2
1	0.0	1.0
2	0.2	0.8
3	0.8	0.2
4	1.0	0.0

Table 5: Different values of w_1 and w_2

The results are shown in Figures 8, 9, 10 and 11.

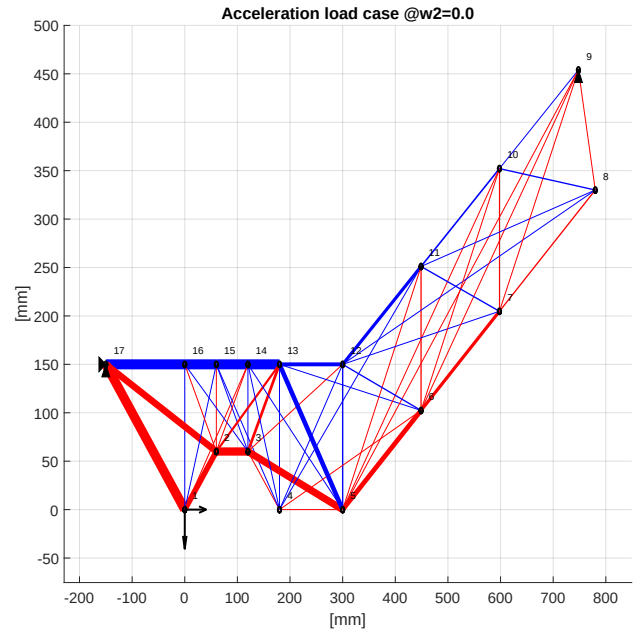
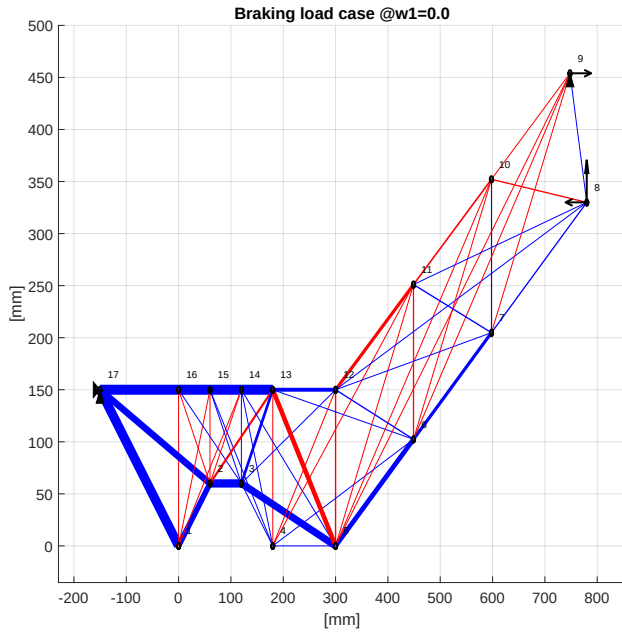


Figure 8: Optimized motorcycle frame for $w_1 = 0.0$ and $w_2 = 1.0$

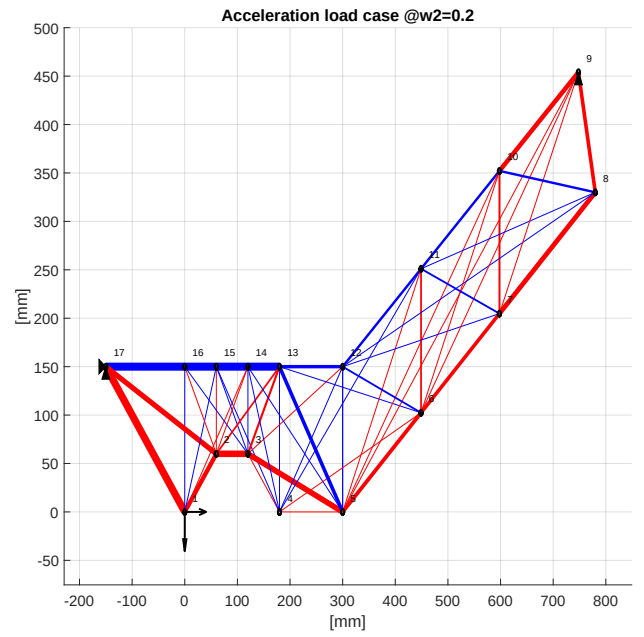
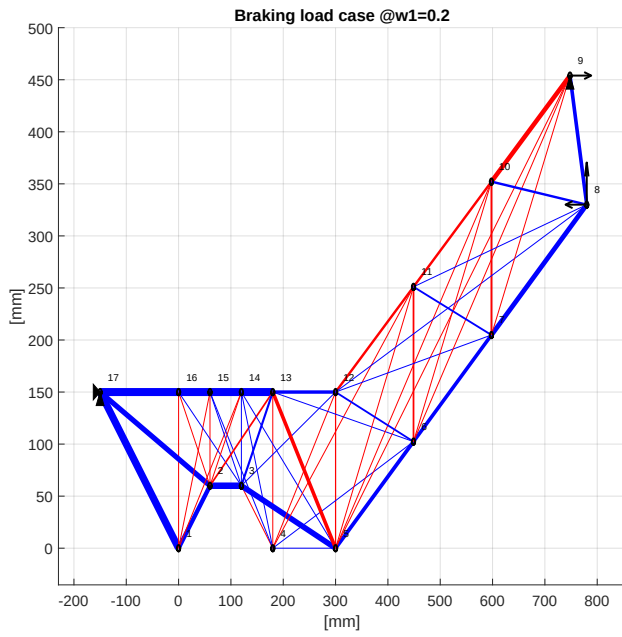


Figure 9: Optimized motorcycle frame for $w_1 = 0.2$ and $w_2 = 0.8$

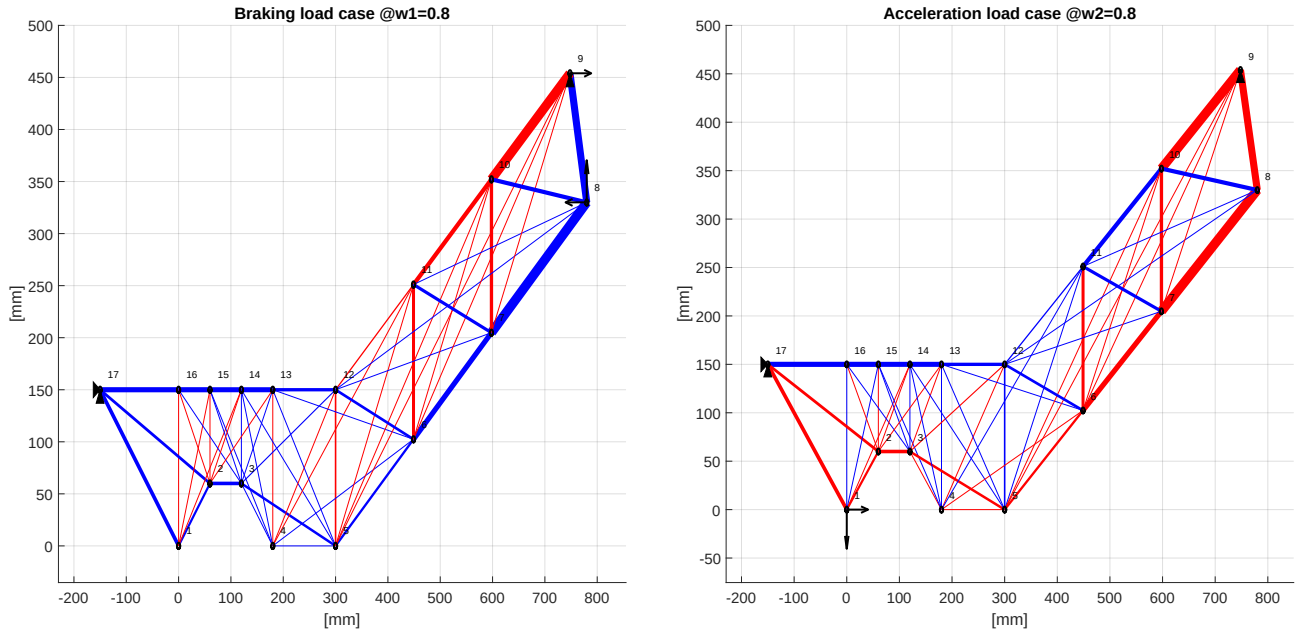


Figure 10: Optimized motorcycle frame for $w_1 = 0.8$ and $w_2 = 0.2$

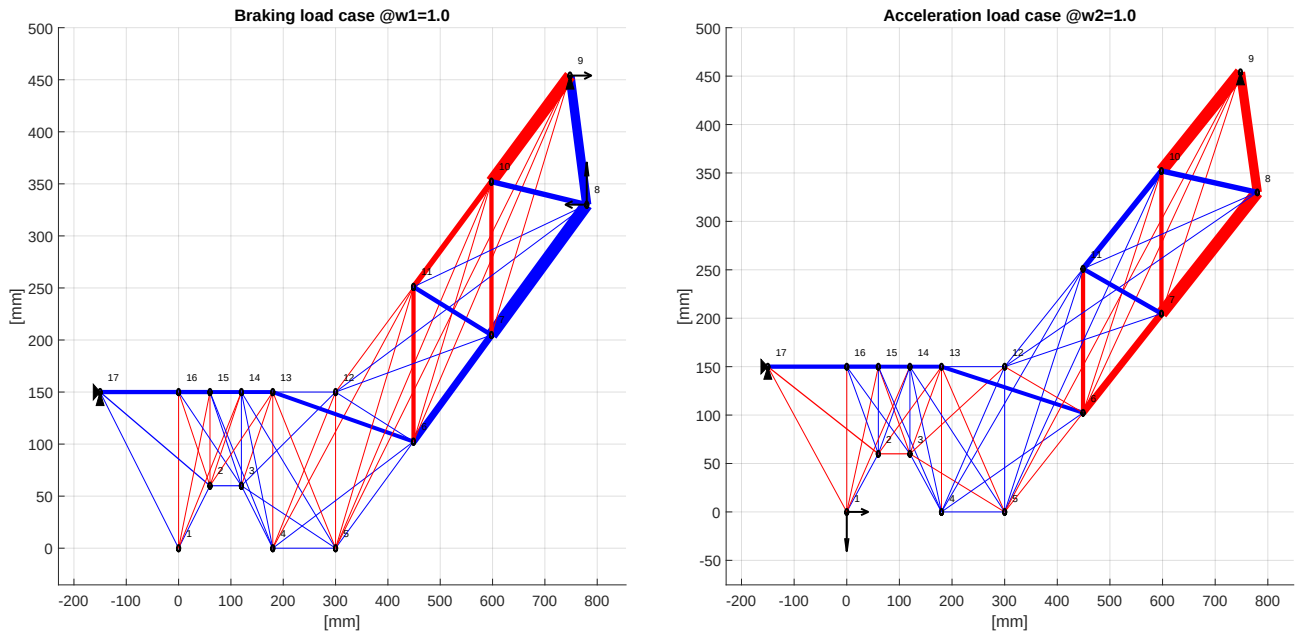


Figure 11: Optimized motorcycle frame for $w_1 = 1.0$ and $w_2 = 0.0$

References

- [1] Anders Klarbring Peter W. Christensen. *An Introduction to Structural Optimization*. Springer Dordrecht, 2010.

A Compliance minimization flowchart

The flowchart in Figure 12 shows the steps to solve a compliance minimization problem. One can refer to Section 2.2.1 for a detailed explanation of the theoretical background.

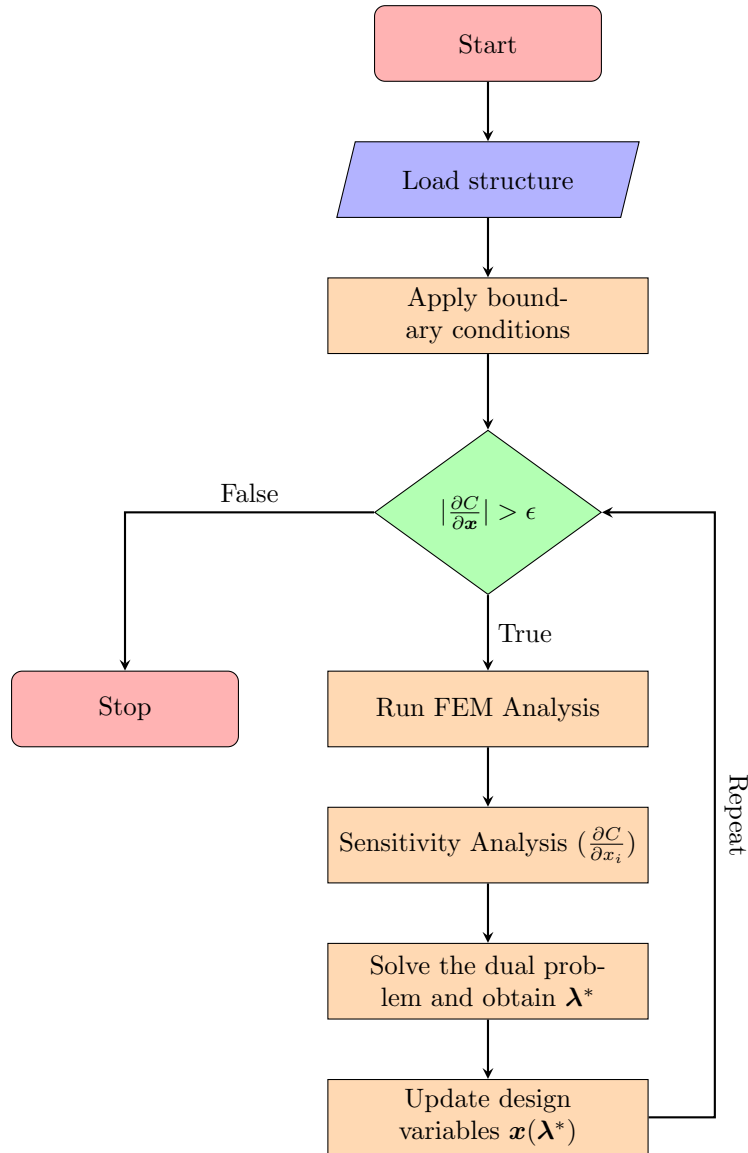


Figure 12: Compliance minimization flowchart

B MATLAB code for the compliance minimization algorithm

The MATLAB code in Listing 2 shows the implementation of the compliance minimization algorithm. One can refer to Section 2.2.1 or section 3.2 for a detailed explanation of the algorithm and the naming conventions used for the variables.

```
1 for k = 1 : N
2
3     if(mod(k, 25) == 0)
4         disp(['Iteration #', num2str(k)]);
5     end
6
7     % FEM Analysis
8     structure = structure.runFEM();
9     results.C(k) = structure.F' * structure.U;
```

```

11 % Sensitivity Analysis
12 for ii = 1:length(structure.elements)
13
14     k0 = structure.elements(ii).getStiffnessMatrix / structure.elements(ii).A;
15     ue = structure.U(structure.getDofIdxs(structure.elements(ii)));
16
17     results.dCdA(k, ii) = - ue' * k0 * ue;
18
19 end
20
21 % Dual problem solution
22 handler = @(lambda) dphi( ...
23     lambda, ...
24     [structure.elements.A]', ...
25     lower_bound, ...
26     upper_bound, ...
27     results.dCdA(k, :)', ...
28     [structure.elements.L]', ...
29     max_volume_constrained);
30
31 % Solve  $\sum(L * A(\lambda)) - V_{\{0\}} == 0$  and compute  $\lambda$ 
32 lambda = fsolve(handler, lambda, optimoptions('fsolve', 'Display', 'off'));
33 [~, A] = handler(lambda);
34
35 % Variables updates
36 for ii = 1:length(structure.elements)
37     structure.elements(ii).A = A(ii);
38 end
39 results.V(k) = A' * [structure.elements.L]';
40 results.alpha(k) = results.V(k) / max_volume;
41 results.A(k, :) = A;
42 results.stress(k, :) = structure.computeStresses();
43
44 % Check convergence condition
45 if(norm(results.dCdA(k, :)) < epsilon)
46     break;
47 end
48
49 end
50
51 %% Functions
52
53 function [y, x] = dphi(lambda, A, lower_bound, upper_bound, dCdA, l_elements,
54     max_volume_constrained)
55
56 lambda(lambda < 0) = 0;
57
58 q0 = -A.^2 .* dCdA;
59 x = sqrt(q0 ./ (lambda * l_elements));
60
61 x(x > upper_bound) = upper_bound;
62 x(x < lower_bound) = lower_bound;
63
64 y = l_elements' * x - max_volume_constrained;
65
66 end

```

Listing 2: MATLAB code for the compliance minimization algorithm